

Project Synopsis: Connected Car AI Support Console

PRE-PROJECT REVIEW SUBMISSION FOR GUIDE APPROVAL

Project Title:	Connected Car AI Support Console
Domain:	Artificial Intelligence, Multi-Agent Systems, Natural Language Processing, RAG, Automotive IoT
Deployment:	Hugging Face Spaces (Dockerized Container)
Orchestration:	LangGraph (StateGraph Workflow)
Core Engine:	Python 3.11/3.12, Google Gemini 2.5 Flash

1. Project Overview & Abstract

Modern passenger vehicles are equipped with telematics control units (TCUs) and mobile app companions (e.g., Tesla App, MyBMW, FordPass) that coordinate with cloud servers, billing systems, and hardware ECUs. When a customer reports a connectivity or remote operation failure (e.g., remote-start failure, pairing disconnects, or navigation routing drops), support engineers must manually trace logs across multiple isolated databases:

- **Customer Relationship Management (CRM)** databases to identify owner details and VIN histories.
- **Telematics Databases** to check signal strengths (RSSI), cellular network status, and battery State of Charge (SoC).
- **Billing & Subscription Databases** to check active entitlements for remote packages.
- **Mechanical & Technical Manuals** to cross-reference diagnostic trouble codes (DTCs) and troubleshooting steps.

The **Connected Car AI Support Console** is an automated, stateful, multi-agent diagnostic platform that uses a Directed Acyclic Graph (DAG) state-machine to automate this investigation. Powered by **LangGraph** and **Google Gemini 2.5 Flash**, the system classifies incoming customer complaints, dynamically executes data-retrieval routes, retrieves troubleshooting advice using a Retrieval-Augmented Generation (RAG) pipeline powered by **ChromaDB**, isolates the root cause with evidence, and generates step-by-step remediation plans. The entire platform is fronted by a dark-themed glassmorphic Streamlit dashboard and deployed inside a secure Docker container on Hugging Face Spaces.

2. Problem Statement & Motivation

As vehicles become software-defined IoT platforms, the volume of software-related support inquiries has scaled exponentially. Debugging these issues presents several friction points:

1. **Siloed Diagnostics:** Customer records, subscription status, and live vehicle pings reside in separate databases. Support agents waste critical time context-switching between tools.
2. **Complex Dependencies:** A single symptom (e.g., "Cannot lock doors via app") could be caused by an expired subscription, a dead key fob battery, a sleeping cellular TCU, or a faulty lock actuator.
3. **Variable Technical Literacy:** Human support representatives require immediate access to updated mechanical manuals and service bulletins.
4. **High Operational Costs:** Complex issues are escalated prematurely to expensive Tier-3 field engineers, delaying resolution.

By orchestrating specialized agents that execute database retrievals conditionally, this system reduces Case Resolution Time (CRT) from hours to seconds, providing clear evidence-backed diagnoses and automated mitigation protocols.

3. Workflow & Agentic DAG

The project uses LangGraph's stateful orchestration to route data sequentially and conditionally across different nodes.

Intent & Planner Agent (Gemini 2.5 Flash) processes the query and maps severity.

- CRM Agent extracts details and maps the VIN.

- Conditional Router triggers:

- Telematics Agent (Checks Signal & Battery status)

- Subscription Agent (Validates Billing Entitlements)

- Knowledge Base RAG Node (Queries manuals from ChromaDB)

- Investigation Agent (Isolates Root Cause & Resolution)

- Safety Gate: Confidence Check ($\geq 70\%$ vs $< 70\%$ Escalate)

4. Shared State Schema (AgentState)

To maintain session context across all agents, the system shares a common mutable state declared in `graph/state.py` using Python's TypedDict notation:

```
class AgentState(TypedDict):
    customer_query: str          # Raw customer complaint description text
    planner_decision: dict       # Dynamic switches: {"crm": bool, "telematics": bool, ...}
    issue_category: str         # Mapped category (e.g. app_pairing, vehicle_start_issue)
    severity: str               # Threat level (low, medium, high, critical)
    confidence: float           # LLM confidence score on classification
    customer_id: str            # Customer account ID
    vehicle_id: str             # Vehicle VIN identifier
    crm_data: dict              # Customer details pulled from CRM
    telematics_data: dict       # Live hardware sensor readings
    subscription_status: str     # Billing package state (active/expired)
    kb_context: str             # Text context retrieved by RAG
    root_cause: str             # Isolated failure origin
    resolution: str             # Actionable remediation steps
    root_cause_confidence: float # Diagnostic confidence score
    evidence_used: list          # Specific parameters proving the root cause
    investigation_steps: list    # Step-by-step pipeline execution logs
```

5. Agent Nodes & Functional Specifications

- 1. Intent & Planner Node:** Evaluates the query to identify domain category and dynamically generates the database queries checklist. Avoids hitting unnecessary endpoints, saving LLM tokens.
- 2. CRM Profile Node:** Fetches owner schemas (e.g. ticket counters, car VIN models) from `crm.json` based on customer ID.
- 3. Telematics Node:** Queries live ECU statuses (cellular online status, network latency, and battery charge levels) if required.
- 4. Subscription Node:** Queries billing database registers to ensure valid service package entitlements.
- 5. Knowledge Base RAG Node:** Queries ChromaDB to retrieve semantically-relevant manual instructions and symptom solutions.
- 6. Investigation Node:** Consolidates and reasons over all state parameters to isolate the root cause, compile evidence, and generate step-by-step remediation plans.

6. Databases & Data Schema

CRM Database (`crm.json`):

```
{"C001": {"name": "Hemanth N", "vehicle_id": "V001", "previous_tickets": 3}}
```

Telematics ECU (`telematics.json`):

```
{"V001": {"online": true, "battery": 78, "network": "good"}}
```

Subscription Entitlements (`subscriptions.json`):

```
{"C001": {"status": "expired"}, "C002": {"status": "active"}}
```

7. Safety Gateways & Governance Guardrails

To prevent incorrect diagnostics or OTA scripts execution on real vehicles, the system implements guardrails: If the isolated root cause confidence is below **70%** or the intent is classified as **unknown**, the system blocks automatic output and instantly routes the issue to human Tier-3 support engineers.

8. Technology Stack

- **Stateful Graph Engine:** LangGraph StateGraph (DAG Framework)
- **Reasoning LLM Model:** Google Gemini 2.5 Flash (optimized for speed, cost, and reasoning)
- **Vector Database (RAG):** ChromaDB (Persistent storage matching manuals with customer queries)
- **Frontend Console:** Streamlit Custom Dashboard (Outfit Google font, customized glassmorphic container stylesheets)
- **Deployment & Containers:** Docker (Python 3.11 base), hosted on Hugging Face Spaces

9. Dockerization & Hugging Face Deployment

The application is fully containerized. A Dockerfile installs dependencies, exposes port 7860 (standard Hugging Face routing target), and runs the Streamlit server. The project is deployed on Hugging Face Spaces using the Docker SDK. The `GOOGLE_API_KEY` credential is managed securely via Hugging Face Repository Secrets.

10. Key Features & Academic Value

- **Audit Trail:** Logs and displays exactly which nodes were visited or skipped.
- **Efficiency:** Planner-driven dynamic routing prevents redundant API calls.
- **Explainability:** Output lists the precise database criteria used as evidence.